Django REST Framework (DRF) 실습 가이드

교육생 여러분, 이번 실습에서는 Django REST Framework(DRF)를 활용하여 기본적인 API 서버를 구축하고, Postman을 통해 CRUD 동작을 테스트합니다. 아래 순서에 따라 코드를 작성하고 실습을 진행해 주시기 바랍니다.

1. Django DRF 서버 생성 및 모델 정의

가장 먼저 실습을 진행할 프로젝트 환경을 구성하고, 게시글 데이터를 저장할 모델을 만듭니다.

1. **프로젝트 설정 및 앱 생성**

```
python -m venv venv
source venv/bin/activate  # Windows: venv\Scripts\activate
pip install django djangorestframework
django-admin startproject myapi .
python manage.py startapp articles
```

2. `myapi/settings.py` **설정 추가**

```
INSTALLED_APPS = [
    ...
    'rest_framework',
    'articles',
]
```

3. **Model 작성 (**`articles/models.py`**)**

```
from django.db import models

class Article(models.Model):
    title = models.CharField(max_length=100)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

4. **마이그레이션 실행**

```
python manage.py makemigrations
python manage.py migrate
```

2. Serializer 작성

Django 모델 데이터를 JSON 형식으로 변환하고 검증하기 위해 Serializer를 작성합니다.

- `articles/serializers.py` **생성 및 작성**

```
from rest_framework import serializers
from .models import Article

class ArticleSerializer(serializers.ModelSerializer):
    class Meta:
        model = Article
        fields = '__all__'
```

3. View 함수 작성

클라이언트의 요청을 처리할 API View 함수를 작성합니다.

- `articles/views.py` **작성**

- ```
  from rest_framework.response import Response
  from rest_framework.decorators import api_view
  from rest_framework import status
  from django.shortcuts import get_object_or_404
  from .models import Article
  from .serializers import ArticleSerializer

  @api_view(['GET', 'POST'])
  def article_list(request):
      if request.method == 'GET':
          articles = Article.objects.all()
          serializer = ArticleSerializer(articles, many=True)
          return Response(serializer.data)

      elif request.method == 'POST':
          serializer = ArticleSerializer(data=request.data)
          if serializer.is_valid(raise_exception=True):
              serializer.save()
              return Response(serializer.data, status=status.HTTP_201_CREATED)

  @api_view(['GET', 'PUT', 'DELETE'])
  def article_detail(request, pk):
      article = get_object_or_404(Article, pk=pk)

      if request.method == 'GET':
          serializer = ArticleSerializer(article)
          return Response(serializer.data)

      elif request.method == 'PUT':
          serializer = ArticleSerializer(article, data=request.data)
          if serializer.is_valid(raise_exception=True):
              serializer.save()
              return Response(serializer.data)

      elif request.method == 'DELETE':
          article.delete()
          return Response(status=status.HTTP_204_NO_CONTENT)
  ```

4. URL 설정 및 설명

작성한 View 함수를 클라이언트가 접근할 수 있도록 URL과 연결합니다.
  ```

1. `myapi/urls.py` 설정

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('articles/', include('articles.urls')),
]
```

2. `articles/urls.py` 생성 및 작성

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.article_list, name='article_list'),
    path('<int:pk>/', views.article_detail, name='article_detail'),
]
```

URL 패턴	Method	설명 (CRUD)
`articles/`	GET	전체 게시글 목록 조회 (Read - List)
	POST	새로운 게시글 생성 (Create)
`articles/<int:pk>/`	GET	특정 게시글 상세 조회 (Read - Detail)
	PUT/PATCH	특정 게시글 내용 수정 (Update)
	DELETE	특정 게시글 삭제 (Delete)

5. Load Data (초기 데이터 불러오기)

실습을 위해 `articles/fixtures/` 폴더에 `articles.json` 파일을 준비하고 더미 데이터를 로드합니다.

```
python manage.py loaddata articles.json
python manage.py runserver
```

6. Postman을 활용한 CRUD 실행 방법

Postman 프로그램을 열고, 로컬 서버(`http://127.0.0.1:8000/`)를 기준으로 아래 요청들을 테스트합니다.

① Create (생성)

- **URL:** `http://127.0.0.1:8000/articles/`
- **Method:** POST
- **설정:** Body 탭 선택 ➔ raw 선택 ➔ JSON 포맷 지정
- **데이터 입력:**

```
{
    "title": "새로운 실습 게시글",
    "content": "DRF Postman 테스트 중입니다."
}
```

- **결과:** Send 버튼 클릭 후 상태 코드 `201 Created` 확인

② Read (조회)

- **전체 목록 조회:** `http://127.0.0.1:8000/articles/` ➔ GET (상태 코드 `200 OK`)
- **단일 상세 조회:** `http://127.0.0.1:8000/articles/1/` ➔ GET (상태 코드 `200 OK`)

③ Update (수정)

- **URL:** `http://127.0.0.1:8000/articles/1/`
- **Method:** PUT
- **설정:** Body 탭 ➔ raw ➔ JSON
- **데이터 입력:**

```
{
    "title": "수정된 게시글 제목",
    "content": "내용이 수정되었습니다."
}
```

- **결과:** Send 버튼 클릭 후 상태 코드 `200 OK` 확인

④ Delete (삭제)

- **URL:** `http://127.0.0.1:8000/articles/1/`
- **Method:** DELETE
- **결과:** Body 입력 없이 Send 클릭 후 `204 No Content` 확인